



Lecture 4:

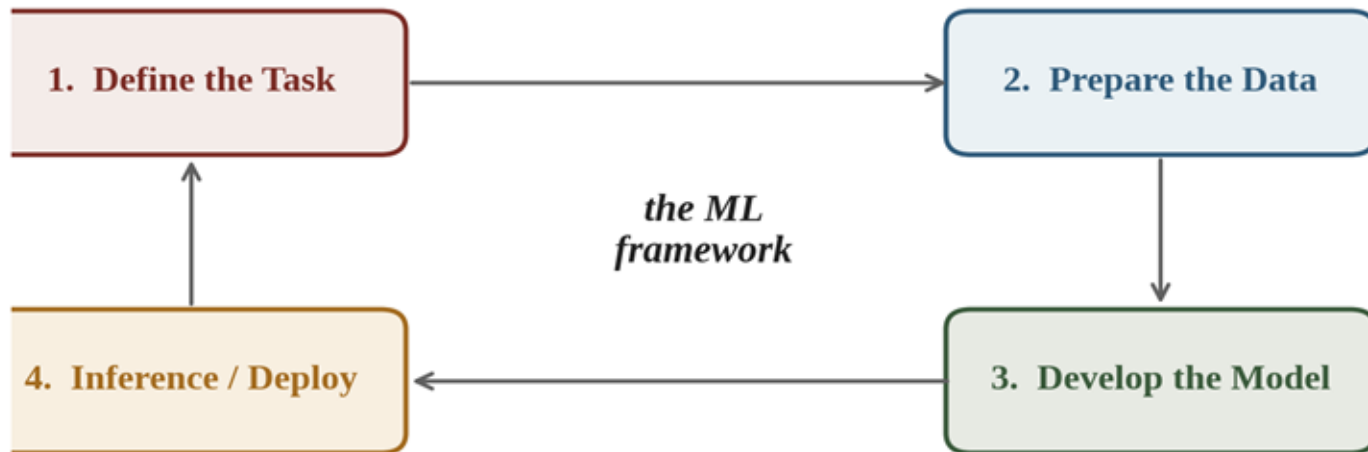
Practical Implementation of Random Forest

The ML framework, common pitfalls, and a worked photo-z case study

Omkar Bait (CosmicAI Fellow)
NRAO, Charlottesville

The ML Framework — One Picture

Every supervised-ML project follows the same four steps. Skipping any leads to trouble.



Today: we revisit each step briefly, then walk through a complete real-world example.

Step-by-Step Recap of the Course

1

Define the Task *(Lec 1b)*

What is X? What is y? Classification or regression? What does success mean (RMSE, accuracy)?

2

Prepare the Data *(Lec 2)*

EDA → missing values → outliers → scaling → encoding → split into train / val / test.

3

Develop the Model *(Lec 3a + 3b)*

Pick a model → beat a baseline → cross-validate → tune hyperparameters → guard against leakage.

4

Inference / Deploy *(Lec 4)*

Final evaluation on test set → uncertainty quantification → apply to unseen real data.

scikit-learn — Your Toolbox

Open-source, BSD-licensed, the de-facto standard for ML in Python.

Datasets & loading

fetch_openml, load_iris, load_digits, plus the I/O helpers in pandas.

Preprocessing

SimpleImputer, KNNImputer, StandardScaler, MinMaxScaler, OneHotEncoder, PCA.

Models

LinearRegression, DecisionTreeClassifier, RandomForestClassifier, GradientBoosting, SVM, MLP.

Pipelines & evaluation

Pipeline, ColumnTransformer, train_test_split, StratifiedKFold, GridSearchCV, cross_val_score.

All-in-one install: `pip install numpy pandas scikit-learn matplotlib jupyter`

Pitfall 1 — Causation vs Correlation

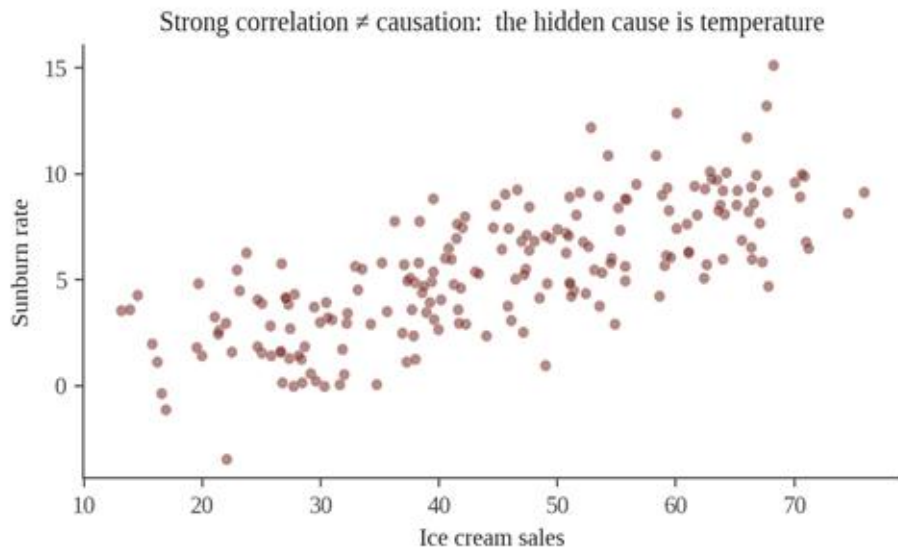
ML finds patterns. It does not know *why* the pattern exists.

Classic example:

Ice cream sales and sunburn rates are strongly correlated.

But eating ice cream doesn't cause sunburn.

Both are driven by a hidden cause: **temperature**.



In astronomy: galaxy color correlates with redshift (and we'll exploit that today). But color does not *cause* redshift.

Don't extrapolate beyond the training range.

Pitfall 2 — Overfitting

A model with too much capacity can memorize the training set noise.

How to spot it

- Training metric → near-perfect.
- Validation / test metric → much worse.
- Predictions look noisy or wildly variable on small data perturbations.
- Feature importances dominated by a single feature.

How to fix it

- Get more training data.
- Reduce model complexity (max_depth, fewer features).
- Add regularization (ccp_alpha, weight decay).
- Cross-validate (Lec 3a) — never trust a single split.
- Use ensembles (bagging, RF) — average out the noise.

Mantra: always report **two numbers:** training metric and test metric. The gap is the diagnostic.

Pitfall 3 — Bias in the Training Data

The model can only learn what is in your training set. Holes in coverage become silent failures.

Selection bias

Training sample isn't a random draw from the deployment population.

→ *Spectroscopic samples are biased toward bright nearby galaxies.*

Label bias

The labels reflect human / pipeline choices, not the underlying truth.

→ *Citizen-science galaxy morphology has inter-rater disagreement.*

Measurement bias

Survey or instrument systematics that change with sky position or epoch.

→ *Different filter calibrations between SDSS and DES create offsets.*

Representation bias

Rare but important sub-populations are under-represented.

→ *AGN, lensed quasars, ultra-faint dwarfs — easy to miss.*

Diagnostic: compare distributions of train vs deploy features (Lec 1b slide on distribution shift).

Pitfall 4 — Generalizability

Will your model still work tomorrow, on data it has never seen?

Domain (covariate) shift

New data is drawn from a different feature distribution (e.g. fainter galaxies, new sky region).

Concept shift

The target relationship itself changes (e.g. K-correction at high z that wasn't in training).

Adversarial / OOD

Outliers and rare events the model has never met (catastrophic photo- z failures).

Defenses: **(1)** always estimate uncertainty alongside predictions; **(2)** monitor input distributions in deployment; **(3)** flag low-confidence predictions for human review.

Part 2 · Case Study

Photometric Redshift Estimation with Random Forest

10 000 SDSS DR6 galaxies · 4 colors · scikit-learn

Reference: [Carliles et al. 2010](#)

Why Photometric Redshifts?

Spectroscopy is the gold standard. But it's slow and expensive.

Spectroscopic z (truth)

Disperse light through a grating, identify emission/absorption lines, measure shifts.

- Accurate to $\sim 10^{-4}$.
- Slow: hours per galaxy on big telescopes.
- Expensive — pre-LSST surveys $\approx 10^6$ spec- z .

Photometric z (estimate)

Use the brightness through a small set of broad filters (u, g, r, i, z).

- Lower accuracy ($\sigma \approx 0.02\text{--}0.05$).
- Fast: one image gives 10^6+ galaxies at once.
- Cheap — LSST will deliver $\sim 10^{10}$ photo- z .

Step 1 — Define the Task

Type	Supervised · regression (target is a continuous scalar).
Inputs (X)	4 colors: uberColorU, uberColorG, uberColorR, uberColorI. (Pre-computed from ugriz magnitudes.)
Target (y)	zSpec — spectroscopic redshift in the range [0.002, 0.38] (low-z galaxy sample).
Success metric	RMSE on z, plus bias $\Delta z = \langle z_{\text{Phot}} - z_{\text{Spec}} \rangle$ and outlier fraction $ \Delta z / (1+z) > 0.15$.

Why colors instead of raw magnitudes? colors are scale-free (insensitive to overall brightness) → cleaner physics.

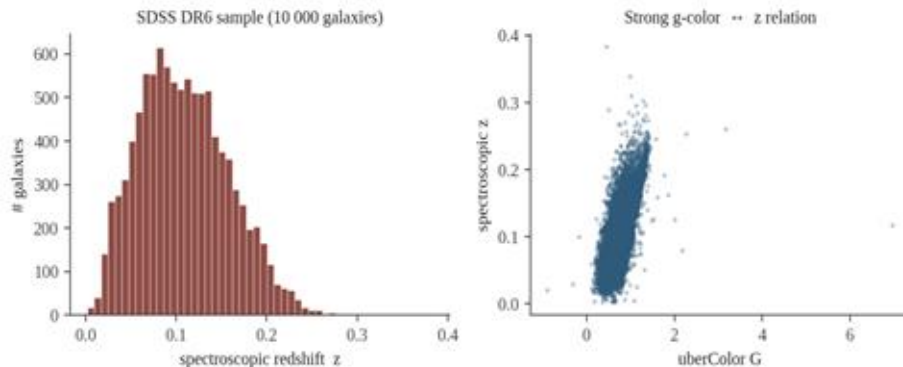
The Dataset — SDSS DR6 Galaxies

10 000 + 10 000 galaxies (train + test).

Drawn uniformly at random from the SDSS DR6
main galaxy sample.

Files we will use:

trainColors10k.txt	(n × 4 features)
trainzSpec10k.txt	(n × 1 target)
testColors10k.txt	(n × 4 features)
testzSpec10k.txt	(n × 1 target)



Features

uberColorU	u − g (extinction-corrected)
uberColorG	g − r
uberColorR	r − i
uberColorI	i − z

Note: the split was done by the dataset authors — already i.i.d. and we treat it as our test holdout.

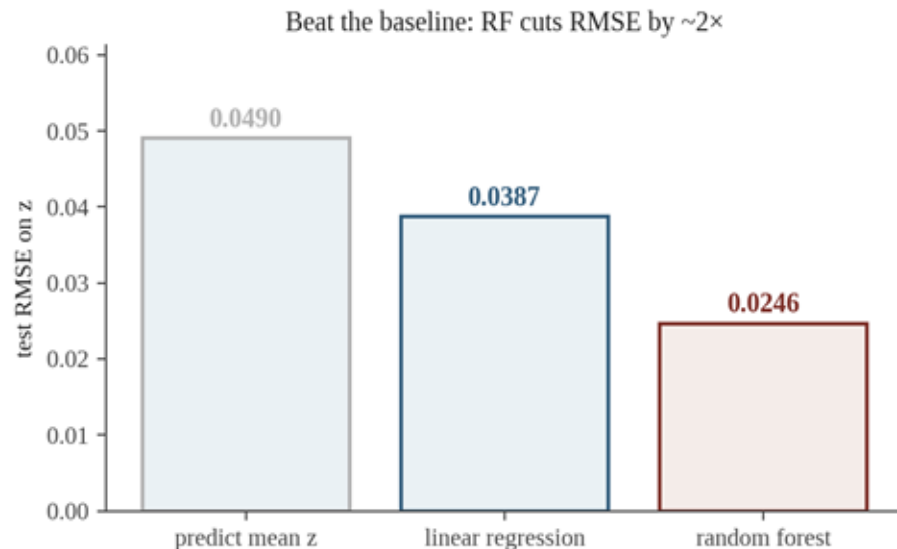
Step 2 — EDA & Baselines

Sanity checks (Lec 1b–2):

- shape (10 000 × 4).
- no missing values.
- redshift distribution skewed to small z .
- colors strongly correlated with z (good).

Two baselines to beat:

- predict the training mean $\langle z \rangle$.
- fit a simple LinearRegression on the 4 colors.



Takeaway: the RF cuts the error of linear regression by $\sim 36\%$ and of the mean baseline by $\sim 50\%$.

Step 3 — Train the Random Forest

Default RF hyperparameters work surprisingly well here — but it's worth knowing what each knob does.

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(
    n_estimators=50,          # # of trees
    max_features="sqrt",     # decorrelate trees
    min_samples_leaf=5,      # soft pre-pruning
    n_jobs=-1, random_state=0,
)
rf.fit(train_colors, train_zspec)
z_phot = rf.predict(test_colors)
```

Tune by cross-validation:

n_estimators: 50 → 500. Beyond ~200 the gain on this dataset is tiny.

max_features: with only 4 features, $\text{sqrt}(4) = 2$ is reasonable.

min_samples_leaf: leaves with only 1 sample tend to over-predict locally; keep ≥ 5 .

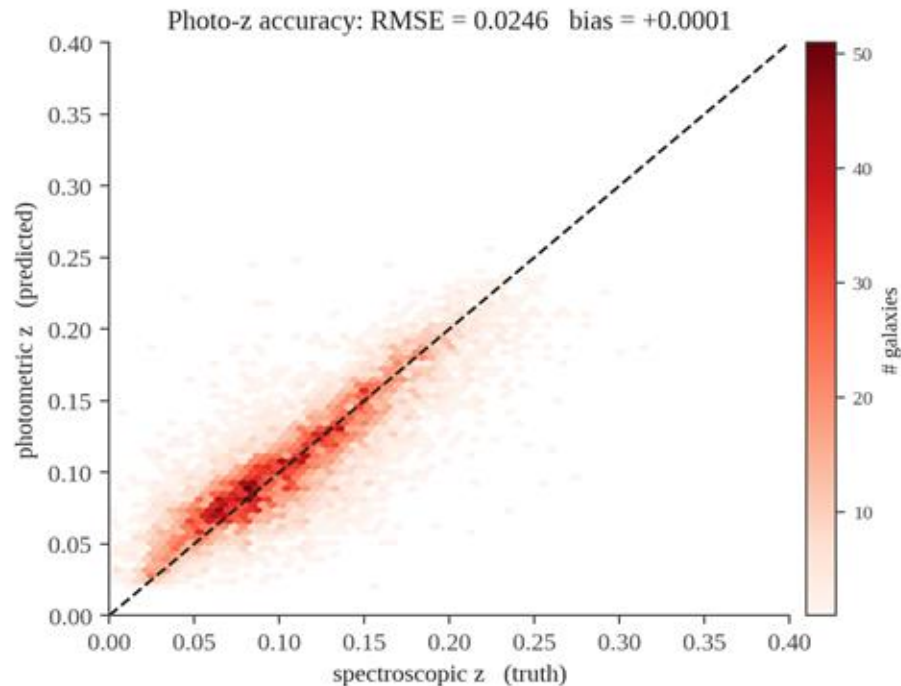
Test result: RMSE = 0.0246, bias = +0.0001, outlier frac < 0.05%.

Result — zPhot vs zSpec

Headline metrics

RMSE	0.0246
Bias $\langle z_{\text{Phot}} - z_{\text{Spec}} \rangle$	+0.0001
MAD $ \Delta z / (1+z)$	0.012
Outlier frac $> 0.15(1+z)$	0.02%

Comfortably under the LSST DESC requirement ($\sigma < 0.05$).



Per-Object Uncertainty

A random forest gives you free uncertainty: the spread of its trees.

$$\sigma(\mathbf{x}) = \sqrt{\text{mean_b} [(\hat{\mathbf{z}}_{\mathbf{b}}(\mathbf{x}) - \hat{\mathbf{z}}_{\text{forest}}(\mathbf{x}))^2] }$$

```
# Aggregate prediction from the forest
z_phot = rf.predict(X_test)

# Individual tree predictions (n_trees, n_samples)
indiv = np.vstack([t.predict(X_test) for t in rf.estimators_])
sigma = np.sqrt(((indiv - z_phot[None,:])**2).mean(axis=0))
```

On this test set: mean $\sigma = 0.021$, somewhat smaller than the actual scatter (RMSE 0.025) → mildly underestimated.

Calibration — Are Our σ 's Honest?

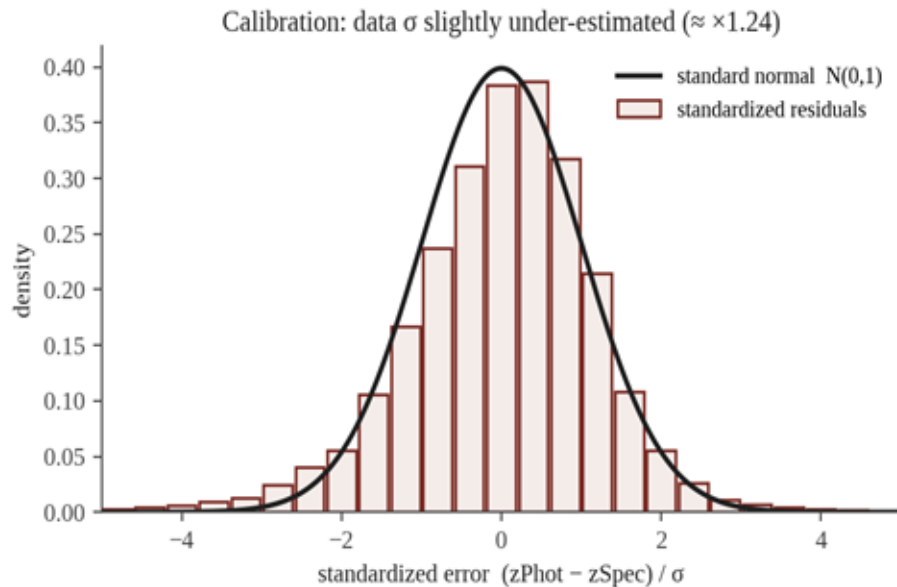
If σ 's were perfectly calibrated, the standardized residuals

$$(z_{\text{Phot}} - z_{\text{Spec}}) / \sigma$$

would follow $N(0, 1)$.

What we find: the empirical std is **1.24**, not 1.0.

Interpretation: the model under-estimates its uncertainty by $\sim 24\%$. Multiply σ by 1.24 for honest reporting.



Wrap-Up & Hand-Off to the Notebook

- 1 Define** supervised regression · colors $\rightarrow$ z · RMSE / bias / outlier frac
- 2 Prepare** 10 000 SDSS galaxies, clean (uber-colors), no missing values
- 3 Model** RF (50 trees, sqrt-features) beats linear regression by 36% RMSE
- 4 Inference** RMSE 0.0246 · bias near zero · per-object σ from the ensemble
- 5 Caveats** σ under-estimated by ~24% · extrapolation beyond $z < 0.4$ unsafe

Now open `RFPPhotoZ_full.ipynb` and walk through it cell by cell. Each section maps to one box of the ML framework — try the suggested extensions at the end.